

1

FORMULARIO DE EQUIVALENCIAS

GNU Octave ↔ Python Cientifico

Equivalencia de comandos entre GNU Octave y Python (NumPy, SciPy, SymPy y Matplotlib).



ENTORNO DE TRABAJO Y AYUDA

Comandos esenciales para gestionar el entorno, obtener ayuda y administrar paquetes.

¿Qué quiero hacer?	GNU Octave	Python
Limpiar consola Borra el contenido de la ventana de comandos.	<code>clc</code>	<pre>import os os.system('cls') # Windows os.system('clear') # Linux / macOS</pre>
Limpiar Workspace Elimina todas las variables del entorno.	<code>clear</code>	<pre># Opción 1: reiniciar kernel # Opción 2: eliminar variables del variable1, variable2, ...</pre>
Cerrar figura actual Cierra la figura activa.	<code>close</code>	<pre>import matplotlib.pyplot as plt plt.close()</pre>
Cerrar todas las figuras Cierra todas las figuras abiertas.	<code>close all</code>	<pre>import matplotlib.pyplot as plt plt.close('all')</pre>
Listar variables Muestra las variables existentes.	<code>who</code>	<pre>for name in dir(): print(name) # Variables definidas</pre>
Información de variables Muestra información detallada de las variables.	<code>whos</code>	<pre>globals() # Diccionario de variables locals() # Variables locales</pre>
Ayuda Muestra ayuda de un comando o función.	<code>help <i>nombre_comando</i></code>	<code>help(nombre_comando)</code>
Listar paquetes instalados Muestra los paquetes disponibles.	<code>pkg list</code>	<pre># Mostrar paquetes instalados import pkg_resources print([d.project_name for d in pkg_resources.working_set])</pre>
Cargar paquete simbólico Carga el paquete para cálculo simbólico.	<code>pkg load <i>symbolic</i></code>	<code>import sympy as sp</code>

2

FORMULARIO DE EQUIVALENCIAS

GNU Octave ↔ Python Científico

Equivalencia de comandos entre GNU Octave y Python (NumPy, SciPy, SymPy y Matplotlib).



GNU Octave



Python



CREACIÓN DE VECTORES Y MATRICES



Comandos para crear, transformar y obtener información de vectores y matrices.

¿Qué quiero hacer?	GNU Octave	Python (NumPy)
 Crear matriz de ceros Matriz de ceros de tamaño m×n.	<code>zeros(m, n)</code>	<code>np.zeros((m, n))</code> # (m filas, n columnas)
 Crear matriz de unos Matriz de unos de tamaño m×n.	<code>ones(m, n)</code>	<code>np.ones((m, n))</code> # (m filas, n columnas)
 Crear matriz identidad Matriz identidad de orden n.	<code>eye(n)</code>	<code>np.eye(n)</code> # (n×n)
 Números aleatorios (0 a 1) Matriz con números aleatorios uniformes en (0,1).	<code>rand(m, n)</code>	<code>np.random.rand(m, n)</code> # (m filas, n columnas)
 Números enteros aleatorios Enteros aleatorios en [a, b].	<code>randi([a, b], m, n)</code>	<code>np.random.randint(a, b+1, (m, n))</code> # incluye b
 Espaciado lineal Vector con n puntos entre a y b.	<code>linspace(a, b, n)</code>	<code>np.linspace(a, b, n)</code> # incluye a y b
 Espaciado logarítmico Vector logarítmico de 10^a a 10^b.	<code>logspace(a, b, n)</code>	<code>np.logspace(a, b, n)</code> # incluye extremos
 Crear arreglo Crea un arreglo (vector o matriz).	<code>[...]</code>	<code>np.array([...])</code> # convierte a ndarray
 Cambiar forma del arreglo Reshape a m filas y n columnas.	<code>reshape(A, m, n)</code>	<code>A.reshape((m, n))</code> # o <code>np.reshape(A, (m, n))</code>
 Tamaño del arreglo Devuelve [filas columnas].	<code>size(A)</code>	<code>A.shape</code> # devuelve tupla (filas, columnas)
 Número de dimensiones Número de dimensiones del arreglo.	<code>ndims(A)</code>	<code>A.ndim</code> # número de dimensiones
 Número total de elementos Cantidad total de elementos.	<code>numel(A)</code>	<code>A.size</code> # total de elementos
 Crear matriz con valores Ejemplo: matriz 2×2.	<code>[a, b; c, d]</code>	<code>np.array([[a, b], [c, d]])</code> # matriz 2×2



3

FORMULARIO DE EQUIVALENCIAS

GNU Octave ↔ Python Científico

Equivalencia de comandos entre GNU Octave y Python (NumPy, SciPy, SymPy y Matplotlib).



GNU Octave



Python



MANIPULACIÓN DE MATRICES E INDEXACIÓN



Comandos para acceder a elementos, realizar operaciones elemento a elemento y operaciones matriciales.

¿Qué quiero hacer?	GNU Octave	Python (NumPy)
<code>:</code>	<code>A(:)</code>	<code>A[:]</code> # todos los elementos
<code>A(i, :)</code>	<code>A(i, :)</code>	<code>A[i, :]</code> # fila i
<code>A(:, j)</code>	<code>A(:, j)</code>	<code>A[:, j]</code> # columna j
<code>A(end, :)</code>	<code>A(end, :)</code>	<code>A[-1, :]</code> # última fila
<code>A(:, end)</code>	<code>A(:, end)</code>	<code>A[:, -1]</code> # última columna
<code>A(i, j)</code>	<code>A(i, j)</code>	<code>A[i, j]</code> # elemento (i, j)
<code>A(i:j, k)</code>	<code>A(i:j, k)</code>	<code>A[i:j+1, k]</code> # fin de índice exclusivo
<code>.*</code>	<code>A .* B</code>	<code>A * B</code> # multiplicación elemento a elemento
<code>./</code>	<code>A ./ B</code>	<code>A / B</code> # división elemento a elemento
<code>.^</code>	<code>A .^ B</code>	<code>A ** B</code> # potencia elemento a elemento
<code>*</code>	<code>A * B</code>	<code>A @ B</code> # producto matricial
<code>A \ b</code>	<code>A \ b</code>	<code>np.linalg.solve(A, b)</code> # solución del sistema



RECUERDE:

Los índices en GNU Octave comienzan en 1, mientras que en Python (NumPy) comienzan en 0. Al convertir código entre ambos lenguajes, ajuste siempre los índices para evitar errores.

Ejemplo:

GNU Octave: `A(1, :)` → Primera fila

Python (NumPy): `A[0, :]` → Primera fila

TEC

Tecnológico
de Costa Rica



Curso CE1111 – Análisis Numérico para Ingeniería
Escuela de Ingeniería en Computadores



Profesor
Dr. Juan Pablo Soto Quirós



Correo
jusoto@tec.ac.cr



GNU Octave



Python



SCRIPTS Y FUNCIONES



Estructura de programas, creación de scripts y funciones y manejo de parámetros y retornos.

¿Qué quiero hacer?	GNU Octave	Python
Crear un script Archivo con instrucciones secuenciales.	<pre>% archivo_script.m instrucciones...</pre>	<pre># archivo_script.py instrucciones...</pre>
Ejecutar script Ejecuta el archivo de script.	<pre>archivo_script</pre>	<pre>python archivo_script.py # o ejecutar desde el IDE</pre>
$f(x)$ Definir función Función con nombre, entradas y salidas.	<pre>function y = mi_func(x) % Código de la función endfunction</pre>	<pre>def mi_func(x): # Código de la función return y</pre>
$f(x,y)$ Función con varios parámetros Función con múltiples entradas.	<pre>function z = f(x, y) ... endfunction</pre>	<pre>def f(x, y): ... return z</pre>
Retornar uno o más valores Devuelve resultados de la función.	<pre>function [a, b] = g(x) ... endfunction</pre>	<pre>def g(x): ... return a, b</pre>
Valor por defecto en argumentos Parámetros opcionales.	<pre>function y = h(x, a = 1, b = 0) ... endfunction</pre>	<pre>def h(x, a=1, b=0): ... return y</pre>
Agregar carpeta al path Incluir carpeta con funciones o scripts.	<pre>addpath('mi_carpeta')</pre>	<pre>import sys sys.path.append('mi_carpeta')</pre>
Listar funciones en archivo Muestra las funciones definidas en un archivo .m.	<pre>which -functions archivo.m</pre>	<pre># No existe equivalente directo # (usar búsqueda en el editor o IDE)</pre>

5

FORMULARIO DE EQUIVALENCIAS

GNU Octave ↔ Python Científico

Equivalencia de comandos entre GNU Octave y Python (NumPy, SciPy, SymPy y Matplotlib).



GNU Octave



Python



ÁLGEBRA LINEAL



Operaciones fundamentales de álgebra lineal: transpuesta, inversa, determinante, rango, factorizaciones, valores y vectores propios.

¿Qué quiero hacer?	GNU Octave	Python (NumPy)
A^T Transpuesta Transpuesta de la matriz A. • Transpuesta simple (sin conjugar) • Transpuesta conjugada (conj. transpuesta)	$A.'$ % transpuesta simple (real) A' % transpuesta conjugada	$A.T$ # transpuesta (no conjugada) $A.conj().T$ # transpuesta conjugada
A^{-1} Inversa Inversa de la matriz A.	$inv(A)$	$np.linalg.inv(A)$
$\det(A)$ Determinante Determinante de la matriz A.	$\det(A)$	$np.linalg.det(A)$
$rank(A)$ Rango Rango de la matriz A.	$rank(A)$	$np.linalg.matrix_rank(A)$
$A*b$ Sistema lineal $Ax = b$ Resuelve el sistema lineal.	$x = A \setminus b$ % mldivide(A,b)	$x = np.linalg.solve(A, b)$
$A \setminus b$ Mínimos cuadrados Solución de mínimos cuadrados $Ax \approx b$.	$x = A \setminus b$ % mldivide(A,b)	$x = np.linalg.lstsq(A, b, rcond=None)[0]$
$A = L*U$ Factorización LU Factorización LU (sin pivoteo).	$[L, U] = lu(A)$	$from scipy.linalg import lu$ $P, L, U = lu(A)$
$A = Q*R$ Factorización QR Factorización QR (completa/full).	$[Q, R] = qr(A)$	$Q, R = np.linalg.qr(A, mode='complete')$
$A = U*D*V'$ Descomposición SVD Valores singulares de A.	$[U, S, V] = svd(A)$ % S es diagonal (matriz)	$U, s, Vh = np.linalg.svd(A, full_matrices=True)$ % s es vector de singulares, $V = Vh.T$
$eig(A)$ Valores y vectores propios Eigenvalores y eigenvectores.	$[V, D] = eig(A)$ % D diagonal, V columnas	$w, V = np.linalg.eig(A)$ # w: valores propios (vector) # V: vectores propios (columnas)
$norm(A)$ Norma matricial Norma 2 (espectral) de la matriz A.	$norm(A, 2)$	$np.linalg.norm(A, 2)$
$trace(A)$ Traza Suma de elementos de la diagonal.	$trace(A)$	$np.trace(A)$



6

FORMULARIO DE EQUIVALENCIAS

GNU Octave ↔ Python Científico

Equivalencia de comandos entre GNU Octave y Python (NumPy, SciPy, SymPy y Matplotlib).



GNU Octave



Python



FUNCIONES NUMÉRICAS



Cálculo de raíces, optimización, integración, derivación numérica y resolución de sistemas no lineales.

¿Qué quiero hacer?	GNU Octave	Python (SciPy / NumPy)
$f(x)=0$ Buscar raíz de una función (escalares) Encuentra x tal que $f(x) = 0$.	<code>fzero(@f, x0)</code> <code>fsolve(@f, x0) % alternativa</code>	<pre>from scipy.optimize import root_scalar root_scalar(f, x0=x0) # (brentq, secant, etc.) from scipy.optimize import fsolve fsolve(f, x0) # alternativa</pre>
$f_1(x)=0$ $f_2(x)=0$ $\vdots$ $f_n(x)=0$ Resolver sistema no lineal Resuelve $F(x) = 0$ para $x \in \mathbb{R}^n$.	<code>fsolve(@F, x0)</code> <code>% F devuelve un vector</code>	<pre>from scipy.optimize import fsolve fsolve(F, x0) # F devuelve un vector</pre>
$\int_a^b f(x) dx$ Integración definida Calcula $\int_a^b f(x) dx$.	<code>quad(@f, a, b)</code> <code>integral(@f, a, b)</code>	<pre>from scipy.integrate import quad quad(f, a, b)</pre>
$\frac{d}{dx} f(x)$ Derivada numérica Aproxima $f'(x)$.	<code>diff(f, x)</code> <code>% diferencias finitas (vector)</code> <code>@(x) diff(f, x) % como función</code>	<pre>from scipy.differentiate import derivative derivative(f, x0, dx=1e-6) # o con NumPy: np.gradient(y, x) # si y = f(x) en una malla</pre>
Minimización sin restricciones Minimiza $f(x)$ escalar.	<code>fminsearch(@f, x0)</code> <code>% Método de Nelder-Mead</code>	<pre>from scipy.optimize import fmin fmin(func=f, x0=x0) # Nelder-Mead</pre>
$a \leftarrow x \rightarrow b$ Minimización en un intervalo Minimiza $f(x)$ en $[a, b]$.	<code>fminbnd(@f, a, b)</code>	<pre>from scipy.optimize import fminbound fminbound(func=f, x1=a, x2=b) # Brent</pre>
$Ax = b$ Resolver sistema lineal Resuelve $Ax = b$.	<code>x = A \ b % mldivide(A,b)</code>	<code>x = np.linalg.solve(A, b)</code>
$y' = f(t, y)$ Ecuaciones diferenciales ordinarias Problema de valor inicial $y' = f(t, y)$.	<code>[t, y] = ode45(@f, [t0 tf], y0)</code> <code>[t, y] = ode15s(@f, [t0 tf], y0)</code>	<pre>from scipy.integrate import solve_ivp sol = solve_ivp(f, (t0, tf), y0, method='RK45') sol = solve_ivp(f, (t0, tf), y0, method='BDF')</pre>
Interpolación Interpola datos (x, y) .	<code>interp1(x, y, xi, 'spline')</code> <code>interp1(x, y, xi, 'linear')</code>	<pre>from scipy.interpolate import interp1d f = interp1d(x, y, kind='cubic') yi = f(xi)</pre>
Distribuciones (ej. Normal) Funciones de distribución y densidad.	<code>pkg load statistics</code> <code>normpdf(x, mu, sigma)</code> <code>normcdf(x, mu, sigma)</code>	<pre>from scipy.stats import norm norm.pdf(x, mu, sigma) norm.cdf(x, mu, sigma)</pre>



7

FORMULARIO DE EQUIVALENCIAS

GNU Octave ↔ Python Científico

Equivalencia de comandos entre GNU Octave y Python (NumPy, SciPy, SymPy y Matplotlib).



GNU Octave



Python



CÁLCULO SIMBÓLICO



Manipulación algebraica exacta: variables simbólicas, simplificación, derivadas, integrales, ecuaciones, series, límites y conversión entre numérico y simbólico.

¿Qué quiero hacer?		GNU Octave	Python (SymPy)
x, y, z	Definir variables simbólicas Crear variables simbólicas.	<pre>syms x y z % variables simbólicas</pre>	<pre>import sympy as sp x, y, z = sp.symbols('x y z') # variables simbólicas</pre>
3 + x²	Expresión simbólica Crear expresiones simbólicas.	<pre>f = 3 + x^2 + sin(x)</pre>	<pre>f = 3 + x**2 + sp.sin(x)</pre>
expand()	Expandir / Simplificar Expandir o simplificar expresiones.	<pre>expand(f) simplify(f)</pre>	<pre>sp.expand(f) sp.simplify(f)</pre>
$\frac{d}{dx} f(x)$	Derivada Derivada simbólica.	<pre>diff(f, x) diff(f, x, 2) % segunda derivada</pre>	<pre>sp.diff(f, x) sp.diff(f, x, 2) # segunda derivada</pre>
$\int_a^b f(x)dx$	Integral indefinida Integración simbólica.	<pre>int(f, x)</pre>	<pre>sp.integrate(f, x)</pre>
$\int_a^b f(x)dx$	Integral definida Integración simbólica en [a, b].	<pre>int(f, x, a, b)</pre>	<pre>sp.integrate(f, (x, a, b))</pre>
Eq(f,g)	Resolver ecuaciones Resolver f(x) = g(x).	<pre>solve(f == g, x)</pre>	<pre>sp.solve(sp.Eq(f, g), x)</pre>
limit	Límites Límite cuando x → a.	<pre>limit(f, x, a)</pre>	<pre>sp.limit(f, x, a)</pre>
$\sum_{i=1}^n a_i$	Sumas Suma simbólica.	<pre>syms i n a S = sum(a(i), i, 1, n)</pre>	<pre>i, n = sp.symbols('i n', integer=True, positive=True) a = sp.Function('a') S = sp.summation(a(i), (i, 1, n))</pre>
$\prod_{i=1}^n a_i$	Productos Producto simbólico.	<pre>syms i n a P = prod(a(i), i, 1, n)</pre>	<pre>i, n = sp.symbols('i n', integer=True, positive=True) a = sp.Function('a') P = sp.product(a(i), (i, 1, n))</pre>
sin(x)	Conversión numérico ↔ simbólico Convertir de numérico a simbólico y viceversa.	<pre>v = sym(pi/4) double(v)</pre>	<pre>v = sp.sympify(sp.pi/4) # simbólico float(v.evalf()) # numérico (aprox.)</pre>
serie	Serie de Taylor Serie de Taylor de f(x) en a hasta orden n.	<pre>taylor(f, x, a, n)</pre>	<pre>sp.series(f, x, a, n).removeO() # hasta (x-a)ⁿ⁻¹</pre>





GNU Octave



Python

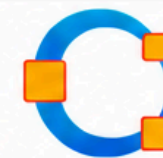


GRÁFICAS 2D CON PYPLOT / MATPLOTLIB



Creación de gráficas 2D: líneas, marcadores, ejes, leyendas, rejillas, subgráficas y personalización.

¿Qué quiero hacer?	GNU Octave	Python (Matplotlib)
Gráfica básica de línea Graficar y vs. x.	<code>plot(x, y)</code>	<code>import matplotlib.pyplot as plt</code> <code>plt.plot(x, y)</code>
Línea con marcadores Especificar estilo y marcador.	<code>plot(x, y, 'o--r')</code>	<code>plt.plot(x, y, 'o--r')</code>
Múltiples curvas en la misma figura Varias series de datos.	<code>plot(x1, y1, 'b', x2, y2, 'r--')</code> <code>legend('y_1', 'y_2')</code>	<code>plt.plot(x1, y1, 'b', x2, y2, 'r--')</code> <code>plt.legend(['y_1', 'y_2'])</code>
Etiquetas y título Agregar etiquetas y título a la gráfica.	<code>xlabel('x')</code> <code>ylabel('y')</code> <code>title('Título')</code>	<code>plt.xlabel('x')</code> <code>plt.ylabel('y')</code> <code>plt.title('Título')</code>
Rejilla Mostrar rejilla en la gráfica.	<code>grid on</code>	<code>plt.grid(True)</code>
Límites de los ejes Definir límites para x e y.	<code>xlim([xmin, xmax])</code> <code>ylim([ymin, ymax])</code>	<code>plt.xlim(xmin, xmax)</code> <code>plt.ylim(ymin, ymax)</code>
Leyenda Mostrar leyenda de las curvas.	<code>legend('c1', 'c2', 'Location', 'best')</code>	<code>plt.legend(['c1', 'c2'], loc='best')</code>
Colores y estilos Personalizar color y tipo de línea.	<code>plot(x, y, 'g-.', 'LineWidth', 2)</code> <code>plot(x, y, 'm:', 'LineWidth', 1.5)</code>	<code>plt.plot(x, y, 'g-.', linewidth=2)</code> <code>plt.plot(x, y, 'm:', linewidth=1.5)</code>
Subgráficas (subplot) Crear varias gráficas en una figura.	<code>subplot(2,1,1); plot(x, y1)</code> <code>subplot(2,1,2); plot(x, y2)</code>	<code>plt.subplot(2, 1, 1); plt.plot(x, y1)</code> <code>plt.subplot(2, 1, 2); plt.plot(x, y2)</code>
Guardar gráfica Guardar la figura en un archivo.	<code>print('-dpng', 'grafica.png')</code> % también: <code>saveas(gcf, 'grafica.png')</code>	<code>plt.savefig('grafica.png', dpi=300, bbox_inches='tight')</code>
Limpiar / cerrar figuras Limpiar figura actual o cerrar todo.	<code>clf()</code> % limpiar figura actual <code>close all</code> % cerrar todas	<code>plt.clf()</code> # limpiar figura actual <code>plt.close('all')</code> # cerrar todas



GNU Octave



Python



PROGRAMACIÓN Y ESTRUCTURAS DE CONTROL



Control del flujo de ejecución: tomar decisiones, repetir acciones y organizar el código.

¿Qué quiero hacer?	GNU Octave	Python
1 <code>if</code> Condicional simple Ejecutar un bloque si se cumple la condición.	<pre>if x > 0 ... endif</pre>	<pre>if x > 0: ...</pre>
2 <code>if</code> <code>else</code> Condicional doble Elegir entre dos opciones.	<pre>if x > 0 ... else ... endif</pre>	<pre>if x > 0: ... else: ...</pre>
3 <code>if</code> <code>elseif</code> <code>else</code> Múltiples condiciones Varias alternativas.	<pre>if x < 0 elseif x == 0 ... else ... endif</pre>	<pre>if x < 0: elif x == 0: ... else: ...</pre>
4 <code>while</code> Bucle while Repetir mientras la condición sea verdadera.	<pre>while condicion ... endwhile</pre>	<pre>while condicion: ...</pre>
5 <code>for i</code> Bucle for Repetir un número de veces o sobre una secuencia.	<pre>for i = 1:10 ... endfor</pre>	<pre>for i in range(1, 11): ... # Python no incluye # el último valor de range()</pre>
6 <code>break</code> Salir del bucle Terminar el ciclo actual.	<pre>break</pre>	<pre>break</pre>
7 <code>continue</code> Saltar a la siguiente iteración Ignorar el resto del ciclo actual.	<pre>continue</pre>	<pre>continue</pre>
8 <code>switch</code> Selección múltiple Según el valor de una variable.	<pre>switch opcion case 1 ... case 2: ... otherwise ... endswitch</pre>	<pre>match opcion: case 1: ... case 2: ... case _: ... # Python 3.10+ (antes de 3.10: usar if-elif-else)</pre>
9 <code>func()</code> Definir una función Crear una función propia.	<pre>function y = f(x) ... endfunction</pre>	<pre>def f(x): ... return y</pre>
10 <code>f(x)</code> Función anónima Función corta de una línea.	<pre>f = @(x) x^2</pre>	<pre>f = lambda x: x**2</pre>
11 <code>%</code> Comentarios Añadir comentarios al código.	<pre>% Este es un comentario</pre>	<pre># Este es un comentario</pre>

⚠ DIFERENCIAS CLAVE

GNU Octave	Python
Usa palabras de cierre (endif, endfor, endwhile)	Usa indentación (espacios)
Índices inician en 1	Índices inician en 0
Potencia con ^	Potencia con **
Función anónima con @	Función anónima con lambda
Comentarios con %	Comentarios con #
Rango 1:10 incluye 10	range(1, 11) no incluye 11



En Python es importante cuidar la indentación: usa 4 espacios por nivel.

